Integração entre back-end, APIs e persistência de dados

Objetivos da aula

- Compreender os aspectos teóricos da integração entre back-end e front-end.
- Analisar como APIs RESTful estruturam essa comunicação.
- Entender versionamento, modelagem de dados, persistência e integração em sistemas distribuídos.

O Papel da API na Arquitetura Moderna

- Em arquiteturas modernas, o front-end não acessa o banco de dados diretamente.
- A comunicação ocorre por meio de APIs.
- A API atua como contrato entre cliente e servidor.

Back e Front como Camadas Independentes

- O front-end é responsável pela interface e experiência do usuário.
- O back-end gerencia regras de negócio e persistência.
- A API conecta essas duas responsabilidades de forma estruturada.

React e Consumo de APIs

- O React consome dados por meio de requisições HTTP.
- Ele utiliza fetch ou bibliotecas como Axios para comunicação.
- A renderização depende das respostas enviadas pelo servidor.

Next.js e Integração com APIs

- O Next.js permite consumo de APIs tanto no cliente quanto no servidor.
- Ele suporta SSR e SSG, ampliando possibilidades arquiteturais.
- A API continua sendo o ponto central de dados.

O que é REST

- REST é um estilo arquitetural baseado em recursos.
- Ele utiliza HTTP como protocolo de comunicação.
- Seu foco é simplicidade e padronização.

Princípio dos Recursos

- Em REST, tudo é tratado como recurso.
- Um recurso representa uma entidade do domínio.
- Ele é identificado por uma URI.

URIs Bem Definidas

- URIs devem representar recursos de forma clara.
- Devem utilizar substantivos, não verbos.
- A estrutura deve ser previsível.

Exemplos Conceituais de URIs

- /usuarios
- /pedidos
- /produtos/10

A URI identifica o recurso, não a ação.

Verbos HTTP

- GET → Buscar
- POST → Criar
- PUT → Atualizar
- DELETE → Remover

Os verbos indicam a operação sobre o recurso.

Statelessness

- APIs REST devem ser stateless.
- Cada requisição contém todas as informações necessárias.
- O servidor não mantém estado de sessão entre chamadas.

Representações

- Os recursos são representados em formatos como JSON.
- JSON tornou-se padrão na comunicação web.
- Ele é leve e facilmente interpretado por JavaScript.

Contrato de Comunicação

- A API define um contrato claro.
- O front-end deve seguir esse contrato.
- Mudanças na API impactam diretamente o cliente.

Versionamento de APIs

- Versionamento evita quebra de compatibilidade.
- Permite evolução sem interromper clientes existentes.
- É prática essencial em sistemas em produção.

Estratégias de Versionamento

- /v1/usuarios
- /v2/usuarios

- Ou via header HTTP.
- A escolha depende da política arquitetural.

Boas Práticas REST

- Consistência nas URIs
- Uso correto dos verbos
- Respostas padronizadas
- Códigos HTTP apropriados

Essas práticas tornam APIs previsíveis.

Códigos de Status HTTP

- 200 → Sucesso
- 201 → Criado
- 400 → Erro do cliente
- 500 → Erro do servidor

Eles comunicam o resultado da operação.

Integração com React

- React realiza requisições assíncronas.
- O estado da aplicação depende das respostas da API.
- Falhas devem ser tratadas adequadamente.

Integração com Next.js

- Next.js pode buscar dados no servidor antes da renderização.
- Isso melhora SEO e performance inicial.
- A API continua sendo fonte de verdade dos dados.

Camada de Persistência

- A persistência armazena dados de forma durável.
- Ela é abstraída da API por meio de serviços.
- A modelagem de dados é etapa fundamental.

Modelagem de Dados

- Modelagem define entidades e relacionamentos.
- É etapa conceitual antes da implementação.
- Impacta desempenho e integridade do sistema.

Bancos Relacionais

- Bancos relacionais utilizam tabelas e relações.
- Garantem integridade referencial.
- São amplamente utilizados em sistemas corporativos.

Exemplos de Bancos Relacionais

- PostgreSQL
- MySQL
- Oracle Database

São baseados no modelo relacional.

Bancos NoSQL

- Bancos NoSQL não seguem modelo relacional tradicional.
- Podem ser orientados a documentos, chave-valor ou grafos.
- Oferecem flexibilidade estrutural.

Exemplo de Banco NoSQL

- MongoDB é orientado a documentos.
- Armazena dados em formato semelhante a JSON.
- É comum em aplicações distribuídas.

ORM

- ORM significa *Object-Relational Mapping*.
- Ele mapeia objetos da aplicação para tabelas do banco.
- Reduz necessidade de SQL manual.

Papel Conceitual do ORM

- O ORM abstrai detalhes da persistência.
- Permite trabalhar com entidades como objetos.
- Facilita integração entre serviço e banco.

Vantagens do ORM

- Produtividade
- Redução de código repetitivo
- Padronização de acesso a dados

Mas pode impactar performance se mal utilizado.

Consistência de Dados

- Consistência garante que dados estejam corretos e íntegros.
- É fundamental em sistemas financeiros e críticos.
- Depende do modelo de transação adotado.

Escalabilidade de Dados

- Escalabilidade pode ser vertical ou horizontal.
- Bancos NoSQL geralmente facilitam escalabilidade horizontal.
- A escolha depende do tipo de aplicação.

Sistemas Distribuídos

- Em sistemas distribuídos, serviços rodam em múltiplos nós.
- A comunicação ocorre por rede.
- Isso exige cuidado com latência e falhas.

Integração entre Serviços

- Serviços podem se comunicar via REST ou mensageria.
- Integração deve ser bem definida e documentada.
- APIs tornam-se contratos formais.

Consistência em Sistemas Distribuídos

- Consistência forte garante dados sincronizados imediatamente.
- Consistência eventual admite atraso na propagação.
- A escolha envolve trade-offs arquiteturais.

Importância do Design da API

- Uma API mal projetada gera acoplamento excessivo.
- Afeta diretamente o front-end.
- Design adequado reduz retrabalho.

Segurança na Integração

- APIs devem proteger dados sensíveis.
- Autenticação e autorização são essenciais.
- Front-end nunca deve confiar apenas na validação do cliente.

Performance da Comunicação

- Reduzir tamanho de payload melhora desempenho.
- Evitar múltiplas chamadas desnecessárias é essencial.
- A arquitetura deve considerar latência.

Documentação de APIs

- Documentação facilita integração.
- Reduz erros de comunicação entre equipes.
- É parte da qualidade arquitetural.

Evolução Controlada

- APIs devem evoluir de forma planejada.
- Versionamento garante compatibilidade.
- Mudanças bruscas prejudicam consumidores.

Integração como Contrato Formal

- Back-end e front-end devem seguir especificações acordadas.
- Esse contrato reduz ambiguidades.
- Permite desenvolvimento paralelo das equipes.

Independência Tecnológica

- Front-end pode evoluir independentemente do back-end.
- React ou Next.js podem ser substituídos sem alterar a API.
- A API é o ponto de desacoplamento.

Integração em Arquiteturas Modernas

- Integração em Arquiteturas ModernasEm microsserviços, múltiplas APIs coexistem.
- Gateway pode centralizar acesso.
- A complexidade aumenta, exigindo padronização.

Importância da Modelagem Teórica

- Antes de implementar, é preciso modelar conceitualmente.
- Entidades mal definidas geram problemas futuros.
- A teoria fundamenta decisões práticas.

Escalabilidade e Integração

- Escalabilidade depende de boa modelagem e design de API.
- Integração mal planejada compromete desempenho.
- Arquitetura deve prever crescimento.

Síntese da Integração

- Front-end consome APIs RESTful.
- Back-end expõe recursos estruturados.
- Persistência é gerenciada por ORM e banco adequado.
- Consistência e escalabilidade sustentam o sistema.

Encerramento

- Você compreendeu o papel das APIs RESTful na integração entre back-end e front-end.
- Analisou versionamento, modelagem de dados e persistência relacional e NoSQL.
- Entendeu como consistência, escalabilidade e integração sustentam sistemas distribuídos modernos.

Essa base teórica permite avaliar arquiteturas com visão estratégica e técnica.

PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.
Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras
modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.